C# MCP Server for SQL Server: Clean Structure

This guide rebuilds your SQL Server MCP server in C# (.NET 10) with separate files, step-by-step comments, and the connection string in appsettings.json. Project folder: `C:\Users\ujjwa\Agentic-AI\MCP-Server-DOTNET`.

Step 1: Folder layout

Inside MCP-Server-DOTNET, create these folders and files. Keep the existing .csproj file.

```
MCP-Server-DOTNET/
  Program.cs
  appsettings.json
  Configuration/
    DatabaseSettings.cs
  Data/
    SqlConnectionFactory.cs
    SchemaRepository.cs
    TableRepository.cs
  Tools/
    SqlTools.cs
```

Create the folders from the terminal:

```
mkdir Configuration, Data, Tools
```

The idea: each file has one job. Program.cs starts the app. The Data files talk to SQL Server. The Tools file is the thin layer that MCP exposes to the AI. Later, when you add CRM write tools, you add a new repository and a new tools file, and nothing else changes.

Step 2: appsettings.json

Create `appsettings.json` in the project root.

```
{
  "Database": {
    "ConnectionString": "Data Source=AYAN-RIYAN;Initial
Catalog=agenticAI_db;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True;",
    "MaxRowLimit": 100
  }
}
```

To change the server or database later, edit only this file.

Step 3: Copy appsettings.json to the output folder

Open your `.csproj` file (the one ending in .csproj) and add this block inside the `<Project>` element, next to the existing `<ItemGroup>` entries. Without it, the app cannot find appsettings.json when it runs.

```
<ItemGroup>
  <None Update="appsettings.json">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </None>
</ItemGroup>
```

Step 4: Configuration/DatabaseSettings.cs

```
namespace McpServerDotNet.Configuration;

/// <summary>
/// Strongly typed settings that map to the "Database" section in
appsettings.json.
/// Keeping settings in a class means the rest of the code never reads raw
config keys.
/// </summary>
public class DatabaseSettings
{
    // The name of the section in appsettings.json. Used when binding the
settings.
    public const string SectionName = "Database";

    // Full SQL Server connection string (Windows authentication in our
case).
    public string ConnectionString { get; set; } = string.Empty;

    // Safety limit so a tool can never return thousands of rows to the AI.
    public int MaxRowLimit { get; set; } = 100;
}
```

Step 5: Data/SqlConnectionFactory.cs

```
using McpServerDotNet.Configuration;
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Options;

namespace McpServerDotNet.Data;

/// <summary>
/// Creates opened SQL connections. Every repository gets connections from
here,
/// so the connection logic lives in exactly one place.
/// </summary>
public class SqlConnectionFactory
{
    private readonly string _connectionString;
```

```
    // The settings are injected by the .NET dependency injection container
(see Program.cs).
    public SqlConnectionFactory(IOptions<DatabaseSettings> options)
    {
        _connectionString = options.Value.ConnectionString;
    }

    /// <summary>Opens and returns a new connection. The caller must dispose
it (use 'await using').</summary>
    public async Task<SqlConnection> OpenAsync()
    {
        var connection = new SqlConnection(_connectionString);
        await connection.OpenAsync();
        return connection;
    }
}
```

Step 6: Data/SchemaRepository.cs

```
using Microsoft.Data.SqlClient;

namespace McpServerDotNet.Data;

/// <summary>
/// Reads schema information: which tables exist and which columns each table
has.
/// It also validates table names, which protects us from SQL injection.
/// </summary>
public class SchemaRepository
{
    private readonly SqlConnectionFactory _factory;

    public SchemaRepository(SqlConnectionFactory factory)
    {
        _factory = factory;
    }

    /// <summary>Returns every user table as 'schema.table', for example
'dbo.Leads'.</summary>
    public async Task<List<string>> GetTablesAsync()
    {
```

```
        var tables = new List<string>();

        await using var conn = await _factory.OpenAsync();
        await using var cmd = new SqlCommand(
            "SELECT TABLE_SCHEMA + '.' + TABLE_NAME " +
            "FROM INFORMATION_SCHEMA.TABLES " +
            "WHERE TABLE_TYPE = 'BASE TABLE' ORDER BY 1", conn);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            tables.Add(reader.GetString(0));
        }

        return tables;
    }

    /// <summary>
    /// Returns column name and data type for one table.
    /// Throws if the table does not exist, so callers get a validation check
for free.
    /// </summary>
    public async Task<Dictionary<string, string>> GetColumnsAsync(string
table)
    {
        var columns = new Dictionary<string, string>();

        await using var conn = await _factory.OpenAsync();
        await using var cmd = new SqlCommand(
            "SELECT COLUMN_NAME, DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS "
+
            "WHERE TABLE_NAME = @table ORDER BY ORDINAL_POSITION", conn);

        // Parameters keep user input separate from the SQL text.
        cmd.Parameters.AddWithValue("@table", table);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            columns[reader.GetString(0)] = reader.GetString(1);
        }

        if (columns.Count == 0)
```

```
        {
            throw new ArgumentException($"Table '{table}' was not found.");
        }

        return columns;
    }
}
```

Step 7: Data/TableRepository.cs

```
using McpServerDotNet.Configuration;
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Options;

namespace McpServerDotNet.Data;

/// <summary>
/// Reads data from a table: sample rows and statistics.
/// Table and column names cannot be SQL parameters, so we validate them
against the
/// real schema first (SchemaRepository) and then wrap them in [brackets].
/// </summary>
public class TableRepository
{
    private readonly SqlConnectionFactory _factory;
    private readonly SchemaRepository _schema;
    private readonly int _maxRowLimit;

    // Data types we can average.
    private static readonly HashSet<string> NumericTypes =
new(StringComparer.OrdinalIgnoreCase)
    {
        "int", "bigint", "smallint", "tinyint", "decimal",
        "numeric", "float", "real", "money", "smallmoney"
    };

    // Data types that cannot be used with MIN, MAX, or DISTINCT.
    private static readonly HashSet<string> SkipTypes =
new(StringComparer.OrdinalIgnoreCase)
    {
        "text", "ntext", "image", "xml", "varbinary", "binary"
```

```
    };

    public TableRepository(
        SqlConnectionFactory factory,
        SchemaRepository schema,
        IOptions<DatabaseSettings> options)
    {
        _factory = factory;
        _schema = schema;
        _maxRowLimit = options.Value.MaxRowLimit;
    }

    /// <summary>Fetches the top N rows of a table. N is clamped between 1
and MaxRowLimit.</summary>
    public async Task<List<Dictionary<string, string?>>> GetRowsAsync(string
table, int limit)
    {
        // Step 1: validate the table name against the real schema.
        await _schema.GetColumnsAsync(table);

        // Step 2: keep the limit inside safe bounds.
        limit = Math.Clamp(limit, 1, _maxRowLimit);

        // Step 3: run the query and turn each row into a dictionary of
column to value.
        var rows = new List<Dictionary<string, string?>>();

        await using var conn = await _factory.OpenAsync();
        await using var cmd = new SqlCommand($"SELECT TOP ({limit}) * FROM
[{table}]", conn);
        await using var reader = await cmd.ExecuteReaderAsync();

        while (await reader.ReadAsync())
        {
            var row = new Dictionary<string, string?>();
            for (int i = 0; i < reader.FieldCount; i++)
            {
                row[reader.GetName(i)] = reader.IsDBNull(i) ? null :
reader.GetValue(i).ToString();
            }
            rows.Add(row);
        }
```

```
        return rows;
    }

    /// <summary>Returns row count plus per-column statistics for a table.
</summary>
    public async Task<Dictionary<string, object?>> GetStatisticsAsync(string
table)
    {
        // Step 1: validate the table and get its columns.
        var columns = await _schema.GetColumnsAsync(table);
        var columnStats = new Dictionary<string, object?>();

        await using var conn = await _factory.OpenAsync();

        // Step 2: total row count.
        long rowCount;
        await using (var countCmd = new SqlCommand($"SELECT COUNT(*) FROM
[{table}]", conn))
        {
            rowCount = Convert.ToInt64(await countCmd.ExecuteScalarAsync());
        }

        // Step 3: statistics for each column.
        foreach (var (column, type) in columns)
        {
            if (SkipTypes.Contains(type)) continue;

            bool isNumeric = NumericTypes.Contains(type);

            // Build one query that returns nulls, distinct, min, max (and
average for numbers).
            var sql =
                $"SELECT SUM(CASE WHEN [{column}] IS NULL THEN 1 ELSE 0 END),
" +
                $"COUNT(DISTINCT [{column}]), MIN([{column}]),
MAX([{column}])" +
                (isNumeric ? $", AVG(CAST([{column}] AS FLOAT))" : "") +
                $" FROM [{table}]";

            await using var cmd = new SqlCommand(sql, conn);
            await using var reader = await cmd.ExecuteReaderAsync();
            await reader.ReadAsync();
```

```
            var stats = new Dictionary<string, object?>
            {
                ["data_type"] = type,
                ["null_count"] = reader.IsDBNull(0) ? 0 :
Convert.ToInt64(reader.GetValue(0)),
                ["distinct_count"] = Convert.ToInt64(reader.GetValue(1)),
                ["min"] = reader.IsDBNull(2) ? null :
reader.GetValue(2).ToString(),
                ["max"] = reader.IsDBNull(3) ? null :
reader.GetValue(3).ToString()
            };

            if (isNumeric)
            {
                stats["average"] = reader.IsDBNull(4) ? null :
Convert.ToDouble(reader.GetValue(4));
            }

            columnStats[column] = stats;
        }

        // Step 4: combine everything into one result.
        return new Dictionary<string, object?>
        {
            ["table"] = table,
            ["row_count"] = rowCount,
            ["columns"] = columnStats
        };
    }
}
```

Step 8: Tools/SqlTools.cs

This is the only file the AI sees. Each method is a tool. The descriptions matter a lot, because the AI reads them to decide which tool to call.

```
using System.ComponentModel;
using McpServerDotNet.Data;
using ModelContextProtocol.Server;

namespace McpServerDotNet.Tools;
```

```
/// <summary>
/// MCP tools for exploring a SQL Server database.
/// The tools stay thin: they only call repositories. No SQL lives here.
/// The repositories are injected into each method by the MCP SDK.
/// </summary>
[McpServerToolType]
public class SqlTools
{
    [McpServerTool(Name = "list_tables")]
    [Description("List all user tables in the database.")]
    public static async Task<List<string>> ListTables(SchemaRepository
schema)
        => await schema.GetTablesAsync();

    [McpServerTool(Name = "describe_table")]
    [Description("Return the column names and data types of a table.")]
    public static async Task<Dictionary<string, string>> DescribeTable(
        SchemaRepository schema,
        [Description("The table name, for example Leads")] string table)
        => await schema.GetColumnsAsync(table);

    [McpServerTool(Name = "fetch_rows")]
    [Description("Fetch the top N rows from a table. The limit is capped for
safety.")]
    public static async Task<List<Dictionary<string, string?>>> FetchRows(
        TableRepository tables,
        [Description("The table name")] string table,
        [Description("Number of rows to return")] int limit = 10)
        => await tables.GetRowsAsync(table, limit);

    [McpServerTool(Name = "table_statistics")]
    [Description("Row count plus per-column statistics: nulls, distinct
values, min, max, and average for numeric columns.")]
    public static async Task<Dictionary<string, object?>> TableStatistics(
        TableRepository tables,
        [Description("The table name")] string table)
        => await tables.GetStatisticsAsync(table);
}
```

Step 9: Program.cs

Replace everything in Program.cs with this. It only wires things together.

```
using McpServerDotNet.Configuration;
using McpServerDotNet.Data;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

// Step 1: create the host. It automatically loads appsettings.json.
var builder = Host.CreateApplicationBuilder(args);

// Step 2: send all logs to stderr. Standard output is reserved for the MCP
protocol,
// so anything printed there would break the connection.
builder.Logging.AddConsole(options =>
{
    options.LogToStandardErrorThreshold = LogLevel.Trace;
});

// Step 3: bind the "Database" section of appsettings.json to
DatabaseSettings.
builder.Services.Configure<DatabaseSettings>(
    builder.Configuration.GetSection(DatabaseSettings.SectionName));

// Step 4: register our data classes so they can be injected into the tools.
builder.Services.AddSingleton<SqlConnectionFactory>();
builder.Services.AddSingleton<SchemaRepository>();
builder.Services.AddSingleton<TableRepository>();

// Step 5: register the MCP server, use stdio, and find every
[McpServerToolType] class.
builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

// Step 6: start and keep running until the client disconnects.
await builder.Build().RunAsync();
```

Step 10: Build and test

Build first. You want zero errors:

```
dotnet build
```

Then test with the Inspector. The `--project` path must point to your project folder:

```
npx.cmd @modelcontextprotocol/inspector dotnet run --project
C:\Users\ujjwa\Agentic-AI\MCP-Server-DOTNET
```

In the Inspector, click Connect, open the Tools tab, click List Tools, and run `list_tables`. You should see your five CRM tables if you created them in agenticAI_db.

Step 11: Connect to Claude Desktop

Open the config file:

```
notepad "$env:APPDATA\Claude\claude_desktop_config.json"
```

Add this server inside the `mcpServers` section, next to the Python one if you added it:

```
"sql-server-dotnet": {
  "command": "dotnet",
  "args": ["run", "--project", "C:\\Users\\ujjwa\\Agentic-AI\\MCP-Server-
DOTNET"]
}
```

Fully quit Claude Desktop from the system tray and reopen it.

Troubleshooting

- **Namespace errors**: the code uses the namespace `McpServerDotNet`. If your project has a different root namespace, either rename the namespaces in each file or add `<RootNamespace>McpServerDotNet</RootNamespace>` to the csproj.
- **Connection string is empty or not found**: check that Step 3 (copy to output) is in the csproj.
- **Login failed**: check the server name AYAN-RIYAN and database name in appsettings.json, and that your Windows user has access.
- **Tool name not found or injection errors**: make sure the ModelContextProtocol package is a recent version; update with `dotnet add package ModelContextProtocol --prerelease`.

- **Never write to Console.Out** in an MCP server over stdio. Use the logger, which we pointed to stderr.

How this maps to your Python server

- `@mcp.tool()` in Python became `[McpServerTool]` in C#.
- `get_connection()` became SqlConnectionFactory.
- `get_table_columns()` became SchemaRepository.
- `fetch_rows` and `table_statistics` logic moved into TableRepository.
- The hardcoded server and database values moved into appsettings.json.

Next: CRM write tools

Once this works, add `Data/LeadRepository.cs` and `Tools/LeadTools.cs` with `create_lead` and `get_leads_by_status`. The structure stays the same: repository for SQL, tools file for MCP.